

Informe Patrón Builder - Proyecto Piedrazul



Universidad
del Cauca

Vigilada Mineducación

INGENIERIA DE SOFTWARE 2

Presentado por:

Juan Camilo Henao Villegas

Juan Pablo Collazos Samboni

Santiago Majé Bonilla

Daniel Alexander Chaguendo

Profesor:

ROBERTO ENCARNACION MOSQUERA

Universidad del Cauca

Facultad de Ingeniería Electrónica y Telecomunicaciones

Ingeniería en Sistemas

Popayán, mayo de 2026

CONTENIDO

1. Fundamentos del Patrón Builder	1
1.1 ¿Qué es el patrón Builder?.....	1
1.2 Estructura del patrón	1
1.3 Diferencia con el Builder de Lombok.....	1
2. Implementación en ProyectoPiedrazul	2
2.1 Contexto del proyecto.....	2
2.2 Clases implementadas	2
CitaBuilder — Abstract Builder	2
CitaProgramadaBuilder — Concrete Builder	2
CitaUrgenteBuilder — Concrete Builder	2
DirectorCita — Director.....	3
2.3 Integración con CitaServiceImpl.....	3
3. Pruebas Unitarias	3
3.1 Estrategia de prueba	3
3.2 Casos de prueba implementados.....	3
3.3 Resultado de la ejecución	4
4. Análisis de Principios SOLID	4
CONCLUSIONES	5

Índice de ilustraciones

No se encuentran elementos de tabla de ilustraciones.

Índice de tablas

No se encuentran elementos de tabla de ilustraciones.

1. Fundamentos del Patrón Builder

1.1 ¿Qué es el patrón Builder?

El patrón Builder es un patrón de diseño creacional cuyo propósito es separar la construcción de un objeto complejo de su representación, de forma que el mismo proceso de construcción pueda crear diferentes representaciones. Fue definido por la Gang of Four (GoF) y responde a la necesidad de construir objetos que requieren múltiples pasos de inicialización o que admiten variantes distintas en su composición.

La motivación central reside en el principio de separación de responsabilidades: ningún objeto debería conocer cómo se construye a sí mismo. En lugar de delegar esa responsabilidad al cliente que lo instancia o de multiplicar constructores con distintas combinaciones de parámetros, el Builder encapsula cada paso en métodos especializados y entrega el control de la secuencia a un Director.

1.2 Estructura del patrón

El patrón Builder se compone de cuatro participantes bien definidos:

- **Producto:** la clase cuya instancia se desea construir. Contiene los campos que se asignarán paso a paso. En el proyecto corresponde a la entidad Cita.
- **Abstract Builder:** clase abstracta que declara los métodos de construcción para cada parte del producto y define el contrato que todos los builders concretos deben cumplir.
- **Concrete Builder:** implementación concreta del Abstract Builder. Cada subclase define cómo ejecutar cada paso y puede producir variantes distintas del mismo producto.
- **Director:** clase que conoce el orden en que deben invocarse los pasos del Builder. Orquesta la construcción sin depender de los detalles de ninguna implementación concreta.

La interacción típica es la siguiente: el Director recibe un Builder concreto, invoca sus métodos en el orden establecido y, al finalizar, el cliente recupera el objeto construido a través del método getCita().

1.3 Diferencia con el Builder de Lombok

Es importante distinguir el patrón Builder GoF del mecanismo generado por la anotación **@Builder** de Lombok, ya presente en la entidad **Cita** del proyecto. El Builder de Lombok es una utilidad de conveniencia que genera una API fluida para construir instancias de una clase concreta, pero no define abstracciones ni permite intercambiar implementaciones.

El patrón GoF, en cambio, introduce una jerarquía de clases: el Abstract Builder define el contrato, los Concrete Builders proveen variantes, y el Director gestiona la secuencia. Esta estructura permite construir una CitaProgramada o una CitaUrgente simplemente cambiando el Builder que recibe el Director, sin modificar ni el Director ni la entidad Cita.

2. Implementación en ProyectoPiedrazul

2.1 Contexto del proyecto

ProyectoPiedrazul es un sistema de gestión de citas médicas implementado con Spring Boot siguiendo una arquitectura en capas. El módulo **Monolito** concentra la lógica del backend: autenticación mediante JWT, gestión de usuarios con roles diferenciados, registro de pacientes y profesionales, disponibilidad semanal con bloqueos e historial clínico asociado a cada cita.

La entidad central sobre la que se aplica el patrón es **Cita**, que referencia a un **Paciente**, un **Profesional**, una fecha y hora, un estado y una marca temporal de creación. Antes de la implementación, la construcción de esta entidad se realizaba directamente en **CitaServiceImpl** mediante la API fluida de Lombok, mezclando la lógica de construcción con la de validación y persistencia.

2.2 Clases implementadas

La implementación introduce cuatro clases en el paquete **model/builder**, más la modificación del servicio existente.

CitaBuilder — Abstract Builder

Clase abstracta que declara los cinco pasos de construcción: **buildPaciente()**, **buildProfesional()**, **buildFechaHora()**, **buildEstado()** y **buildFechaCreacion()**. También provee **iniciarNuevaCita()**, que instancia el objeto Cita sobre el que trabajarán los métodos abstractos, y **getCita()**, que devuelve el producto construido.

```
public abstract class CitaBuilder {
    protected Cita cita;
    public void iniciarNuevaCita() { cita = new Cita(); }
    public Cita getCita() { return cita; }
    public abstract void buildPaciente(Paciente paciente);
    public abstract void buildProfesional(Profesional profesional);
    public abstract void buildFechaHora(ZonedDateTime fechaHora);
    public abstract void buildEstado(EstadoCita estado);
    public abstract void buildFechaCreacion();
}
```

CitaProgramadaBuilder — Concrete Builder

Implementa los cinco pasos del contrato abstracto para construir una cita en estado **programada**. Asigna cada campo directamente sobre la instancia que hereda del Abstract Builder. La fecha de creación se establece automáticamente con **ZonedDateTime.now()** en **buildFechaCreacion()**, garantizando que toda cita quede auditada desde el momento de su construcción.

CitaUrgenteBuilder — Concrete Builder

Segunda implementación concreta del Abstract Builder, diseñada para citas urgentes. Actualmente produce el mismo estado **programada**, pero su existencia como clase independiente permite que en el futuro se diferencie su comportamiento sin modificar **CitaProgramadaBuilder** ni el Director.

DirectorCita — Director

Orquesta el proceso completo de construcción. Recibe cualquier implementación de **CitaBuilder** a través de **setCitaBuilder()** y, cuando se invoca **construirCita()**, ejecuta los pasos en el orden correcto. El cliente obtiene el producto terminado con **getCita()**.

```
public void construirCita(Paciente p, Profesional pr, ZonedDateTime fh) {
    citaBuilder.iniciarNuevaCita();
    citaBuilder.buildPaciente(p);
    citaBuilder.buildProfesional(pr);
    citaBuilder.buildFechaHora(fh);
    citaBuilder.buildEstado(EstadoCita.programada);
    citaBuilder.buildFechaCreacion();
}
```

2.3 Integración con CitaServiceImpl

El método **agendarCita()** de **CitaServiceImpl** fue modificado para delegar la construcción de la entidad al patrón Builder. La lógica de validación de disponibilidad permanece inalterada; únicamente se reemplazó la invocación directa de **Cita.builder()** de Lombok por la siguiente secuencia:

```
DirectorCita director = new DirectorCita();
director.setCitaBuilder(new CitaProgramadaBuilder());
director.construirCita(paciente, profesional, dto.getFechaHora());
Cita cita = director.getCita();
```

Con este cambio, **CitaServiceImpl** deja de conocer cómo se construye una **Cita**. Para soportar citas urgentes basta con cambiar **CitaProgramadaBuilder** por **CitaUrgenteBuilder** sin tocar ningún otro componente del servicio.

3. Pruebas Unitarias

3.1 Estrategia de prueba

Las pruebas unitarias del patrón Builder se ubican en **CitaBuilderTest**, dentro del paquete de prueba **model/builder**. Se utilizan JUnit 5 y objetos stub manuales, sin necesidad de Mockito, dado que las clases Builder no tienen dependencias externas.

3.2 Casos de prueba implementados

- `citaProgramadaBuilderDebeAsignarTodosLosCampos`: verifica que `CitaProgramadaBuilder` asigna correctamente paciente, profesional, fecha y hora, estado programada y fecha de creación.
- `citaUrgenteBuilderDebeAsignarEstadoProgramada`: confirma que `CitaUrgenteBuilder` establece el estado programada y genera la fecha de creación.
- `directorConCitaProgramadaBuilderDebeProducirCitaCompleta`: valida que el Director, usando `CitaProgramadaBuilder`, produce una Cita con todos sus campos correctamente asignados.
- `directorConCitaUrgenteBuilderDebeProducirCitaCompleta`: valida el mismo contrato del Director con `CitaUrgenteBuilder` como builder concreto.
- `directorDebePermitirCambiarBuilderEnTiempoDeEjecucion`: prueba que el Director puede recibir distintos builders sucesivamente y producir instancias independientes.
- `iniciarNuevaCitaDebeReemplazarInstanciaAnterior`: garantiza que invocar `iniciarNuevaCita()` siempre produce una nueva instancia, evitando reutilizar objetos previos.

3.3 Resultado de la ejecución

La ejecución mediante el comando **`mvnw test -Dtest=CitaBuilderTest`** produce el siguiente resultado:

```
-----
T E S T S
-----
Running com.piedrazul.gestioncitasmedicas.model.builder.CitaBuilderTest
Tests run: 6, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.245 s -- in com.piedrazul.gestioncitasmedicas.model.builder.CitaBuilderTest

Results:

Tests run: 6, Failures: 0, Errors: 0, Skipped: 0

-----
BUILD SUCCESS
-----
Total time: 16.188 s
Finished at: 2026-05-11T16:38:27-05:00
```

4. Análisis de Principios SOLID

La implementación del patrón Builder contribuye directamente al cumplimiento de varios principios SOLID:

- Principio de Responsabilidad Única (SRP): `CitaServiceImpl` ya no es responsable de construir la entidad Cita. Esa responsabilidad reside ahora exclusivamente en los builders concretos, mientras el servicio se ocupa de la validación y la persistencia.
- Principio Abierto/Cerrado (OCP): para añadir un nuevo tipo de cita basta con crear un nuevo Concrete Builder que extienda `CitaBuilder`, sin modificar el Director ni el servicio.
- Principio de Inversión de Dependencias (DIP): `CitaServiceImpl` depende de la abstracción `DirectorCita` y del contrato `CitaBuilder`, no de ninguna implementación concreta.
- Principio de Segregación de Interfaces (ISP): `CitaBuilder` define únicamente los métodos necesarios para la construcción de una cita, sin obligar a las implementaciones a depender de operaciones ajenas.

CONCLUSIONES

La implementación del patrón Builder en ProyectoPiedrazul demuestra cómo una correcta separación de responsabilidades mejora la cohesión y reduce el acoplamiento. El servicio **CitaServiceImpl** delega la construcción de la entidad al Director, que a su vez delega los detalles al Builder concreto, produciendo una cadena de responsabilidades clara y localizable.

La distinción entre el Builder GoF y la anotación **@Builder** de Lombok resulta especialmente relevante en un contexto académico: ambos facilitan la creación de objetos, pero solo el primero introduce las abstracciones y jerarquías de clases que permiten intercambiar variantes sin modificar el código cliente.

La extensibilidad del diseño queda demostrada por la presencia de **CitaUrgenteBuilder**: su incorporación no modifica ninguna clase existente, sino que añade una nueva implementación del contrato abstracto. Este es precisamente el comportamiento esperado de un sistema que cumple el Principio Abierto/Cerrado.

Las seis pruebas unitarias ejecutadas con éxito garantizan que tanto los builders individuales como la orquestación del Director producen instancias correctas y reproducibles de la entidad Cita, independientemente de qué variante concreta se utilice.